

Shell Scripts for Automation

| Running Setup Commands Automatically

The Pain of Repeating Commands

In earlier lectures, you typed several commands by hand before the server would even start: install PHP, install MySQL, start MySQL, then run the server.

- Typing the same commands every time is slow, and it is easy to forget a step or make a typo.
- What if we could save that list of commands in a file, and run the file instead?

What Is a Shell Script?

A shell script is just a text file that lists commands, to be run in order — the same commands you would otherwise type by hand, one after another.

- `#!/usr/bin/env bash` at the top of the file tells the computer "run this file using bash."
- `./script_name.sh` runs the file (`./` means "look in this folder").

ensure_install_packages

.sh

```
cd 6_RestAPI/api  
bash ./ensure_install_packages.sh
```

Or, make the script runnable and execute it.

```
cd 6_RestAPI/api  
chmod a+x ./ensure_install_packages.sh  
./ensure_install_packages.sh
```

What this one command does, in plain terms:

```
Installing PHP...  
Installing MySQL...  
Starting MySQL...  
Verifying installation...
```

The First Time Takes Longer

- Downloading PHP and MySQL for the first time can take a few minutes. This is normal.
- The script works the same way on macOS and Ubuntu/WSL2. It checks which one you are using automatically.
- If a step was already done before, the script skips it instead of repeating it.

Password Prompts Are Normal

- Your computer may ask for your **login/admin password** while installing — this is your computer's own password, not a database one.
- A later step may separately ask for a **MySQL root password** — read each prompt carefully, since these two passwords are not the same thing.

Four Things the Install Script Does (1/2)

1. **Installs PHP** — the language our server code is written in.
2. **Installs MySQL** — the database.

Four Things the Install Script Does (2/2)

3. **Starts MySQL** — a database only works while it is running in the background. This kind of background program is called a *service*.
4. **Checks that everything works** before saying it is done.

`./start_server.sh`

```
bash ./start_server.sh
```

- Starts the PHP server at `http://localhost:8080`.
- Keep this terminal window open while you are using the server.
- Closing the window, or pressing `Ctrl+C`, stops it.

JSON vs. MySQL

```
./start_server.sh          # store data in a JSON file (default)  
./start_server.sh --pdo    # store data in a MySQL database
```

- **PDO** is the PHP tool that connects our code to MySQL.
- `--pdo` is called an **option**: extra text after the script name that changes what the script does.

Same URLs, Different Storage

- The URLs, HTTP methods, and request data stay exactly the same in both modes.

- Only the storage handler changes:

`handlers.php` for JSON, `handlers_pdo.php` for MySQL.

See It Work in the Browser

```
http://localhost:8080/student_api_test.html
```

Warning: Do not open this file directly from your file manager (`file:///...`). It must be opened through the address above, while `start_server.sh` is running, or the browser will block the requests.

Clearing Old Test Data:

reset_db.sh

```
./reset_db.sh          # empty the JSON file  
./reset_db.sh --mysql  # empty both JSON and MySQL data
```

Leftover data from a previous test run can cause confusing errors. For example, you might see "this email already exists" when you did not expect it.

reset_db.sh clears out old test data only. It does not touch whether PHP or MySQL are installed.

Stopping the Server:

stop_server.sh

```
./stop_server.sh          # stop the PHP server and MySQL
./stop_server.sh --php    # stop only the PHP server
./stop_server.sh --mysql  # stop only MySQL
```

Always stop the PHP server when you are done for the day. If you don't, the next **./start_server.sh** may fail with an error like "port already in use."

Options for

`start_server.sh`

Option	What it changes
<code>--pdo</code>	Use MySQL instead of the JSON file
<code>--no-reset</code>	Keep existing data instead of clearing it

`--mysql` works the same as `--pdo` here — either one switches to MySQL.

Options for `reset_db.sh`

```
./reset_db.sh          # empty the JSON file  
./reset_db.sh --mysql  # empty both JSON and MySQL data
```

The same word `--mysql` means something slightly different here. For `reset_db.sh` it does not switch modes — it just decides what gets cleared.

Options for `stop_server.sh`

Option	What it changes
<code>--php</code>	Stop only the PHP server
<code>--mysql</code>	Stop only MySQL

Safe Starts with `--no-reset`

You can safely use `--no-reset` at any time — even on your very first run:

- **Defensive setup:** The script automatically checks and prepares the required storage (`data/students.json` or the MySQL `students` table) if it doesn't exist yet.
- **Preserves your data:** If records already exist, `--no-reset` leaves them completely intact.
- If you ever want a clean slate, simply omit `--`

How to Tell If It Worked

Every script prints out what it is doing as it goes:

```
Starting MySQL service...  
MySQL is running and ready.
```

The setup, start, and reset scripts stop when an important step fails, and print a line starting with

ERROR: .

Reading an Error Message

```
ERROR: PHP PDO extension is missing.
```

The **ERROR:** line tells you what failed. It does not tell you how to fix it, but knowing which single piece failed is a good place to start.

Last Resort: Reinstall PHP and MySQL

If PHP or MySQL somehow got installed incorrectly, and nothing above fixes it, this script can help.

```
./ensure_delete_packages.sh
```

This is a **recovery step**, not part of the normal routine — try `./ensure_install_packages.sh` first.

IMPORTANT

Last Resort: What Gets Deleted

Warning: This deletes every local MySQL database managed by Homebrew/APT, not only `studentdb`.

Because it deletes real software, it asks you to type the exact word `DELETE` before doing anything.

Last Resort: What Might Not Get Deleted

- Software installed a different way can still be left behind.
- It does not reliably stop a PHP server you started yourself in another terminal. Use `stop_server.sh` for that first.

IMPORTANT

Normal Use vs. Recovery

Normal, day-to-day use:

1. `./start_server.sh` (installs + starts, if needed)
2. ... test in the browser ...
3. `./reset_db.sh` (clear old data, if needed)
4. `./stop_server.sh` (stop when done)

Only if the environment is badly broken:

`./ensure_delete_packages.sh` -> `./ensure_install_packages.sh`

Deleting everything is the *exception*, not the first step.

Appendix

| How the Scripts Share Code

Three of the scripts — `reset_db.sh`, `start_server.sh`, and `stop_server.sh` — start with a line like this:

```
source "$(dirname "$0")/ensure_install_packages.sh"
```

`ensure_install_packages.sh` defines reusable functions (start MySQL, stop MySQL, stop PHP, ...). `source` loads those functions into the current

| A Dedicated Account

Instead of using the powerful MySQL `root` account, the install script creates one limited account just for this course:

- Database: `studentdb`
- Username: `ase230`
- Password: `ase230pass` (fixed, for local development only)

Appendix: Why a Dedicated Account?

- Normally, `database.php` connects as `ase230`, whose privileges are limited to `studentdb`.
- `database.php` does include a fallback to `root` if that connection fails — mainly to help a partially broken setup still work, not as a security guarantee to rely on.

Coming Next

With the environment automated, the next lecture builds the REST API server itself. It covers routing, HTTP methods, and JSON responses, using `start_server.sh` to launch it without thinking about PHP or MySQL setup again.